

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

CO3 Assessment Tool 2 – Code Review & Repository Evaluation Report

Course Code	CSA1016
Course Title	Software Engineering
CO Assessed	CO3 – Build full-stack applications with an emphasis on containerization, version control, and collaborative development
Project Title	Intelligent Water Distribution Monitoring and Leakage Detection Platform
Total Marks	25

INDEX

S.No.	Section	Page No.
1	Problem Overview	3
2	Repository Organization	4
3	Code Quality Review	6
4	Version Control Evaluation	8
5	Code Testing and Validation	10
6	Repository Documentation	11
7	System Architecture & Component Design	13
8	Code Review Findings and Recommendations	15
9	Conclusion	17

1. Problem Overview

1.1 Assigned Problem Statement

Municipal water utilities and large campus or industrial sites typically discover pipeline leaks only after a visible surface break, a customer complaint, or a sharp rise in a monthly billing cycle, by which point substantial “non-revenue water” has already been lost. Manual inspection walks along kilometres of buried pipework are slow, expensive, and inherently reactive rather than preventive. The assigned problem statement requires the design and implementation of an Intelligent Water Distribution Monitoring and Leakage Detection Platform that continuously ingests flow and pressure telemetry from distributed sensor nodes placed along a pipe network, automatically detects the statistical signature of a leak or burst in near real time, and presents operators with an actionable, map-based alert that narrows down the likely affected segment.

The platform therefore combines three concerns that are normally handled by separate tools: an IoT telemetry-ingestion pipeline, a time-series anomaly-detection engine tuned to the physical behaviour of water networks (a leak manifests as a simultaneous flow increase and pressure drop on the segment feeding it), and an operational dashboard that a non-technical field team can act on.

1.2 Implemented Solution – Summary

The implemented solution is a full-stack IoT application composed of ESP32-based sensor nodes (flow meter + pressure transducer per monitored segment) that publish readings over MQTT to a Mosquitto broker secured with per-device credentials and TLS. A Node.js/Express Ingestion Service subscribes to the broker, validates and normalizes each payload, and writes it to TimescaleDB (a time-series extension of PostgreSQL). A companion Python Leak Detection Engine consumes the same stream, maintains a rolling statistical baseline (EWMA / Z-score) per segment, and cross-checks flagged points with an Isolation Forest model trained on historical readings to reduce false positives before raising an alert.

A React.js dashboard, built around a Leaflet map of the pipe network, renders each segment as a coloured polyline (green/amber/red) driven by live WebSocket updates, alongside time-series charts and an alert queue supporting acknowledge/resolve workflows. Containerization was achieved using Docker so that the broker, database, ingestion service, detection service, and dashboard run as isolated, reproducible services brought up with a single docker compose up command, satisfying the CO3 emphasis on containerization alongside version control and collaborative development.

1.3 Project Objectives

- Continuously ingest flow and pressure telemetry from distributed sensor nodes with sub-second end-to-end latency from publish to dashboard update.
- Automatically detect leaks and bursts per pipe segment using a combination of statistical and machine-learning anomaly detection, minimizing false positives from normal demand fluctuation.
- Visualise network health on an interactive, geo-referenced map with real-time status colouring of segments.
- Generate, queue, and route alerts to operators, with acknowledgement and resolution tracking and a full audit trail.
- Estimate the likely leak location by comparing pressure-drop magnitude between adjacent sensor nodes on the same segment.

- Package the application using Docker and automate build/test/deploy through a CI/CD pipeline.
- Maintain a clean, well-documented Git repository that supports collaborative development practices.

1.4 Key Functionalities

Module	Functionality
Sensor Ingestion Service	Subscribes to MQTT topics per segment, validates payload schema, writes normalized readings to TimescaleDB.
Leak Detection Engine	Rolling statistical baseline plus Isolation Forest model; computes an anomaly confidence score per segment.
Alerting & Notification	Creates alert records, pushes them to the dashboard over WebSocket, and can forward to an email/SMS webhook.
Network Map Dashboard	Leaflet-based map of pipe segments and sensor nodes, colour-coded live by status.
Historical Analytics	Time-series charts of flow/pressure, leak history log, and estimated water-loss reporting.
Authentication & Roles	JWT-based auth with per-user role assignment (Admin, Operator, Viewer).

Table 1.1 – Key functional modules of the platform

1.5 Target Users and Use Cases

- Municipal water-utility operations teams monitoring a city-wide or ward-level distribution network.
- Campus or industrial facilities-management teams monitoring an internal plumbing and irrigation network.
- Field technicians who need to narrow down a likely leak location before a physical dispatch.
- Sustainability and compliance teams tracking non-revenue water loss trends over time.

These use cases were used throughout development to validate design decisions – for example, the requirement that a field technician be shown a ranked list of candidate segments rather than a single point directly motivated storing per-node pressure history rather than only the latest reading, so that the location estimator can compare a rolling window on both sides of a segment.

2. Repository Organization

2.1 Repository Structure

The repository follows a monorepo layout that separates the ingestion service, the detection engine, the dashboard, infrastructure assets, tests, and documentation into clearly named top-level folders. This structure was chosen over multiple repositories because the ingestion payload schema, the detection engine's expected fields, and the dashboard's typed API client must change together, and a monorepo makes such cross-cutting changes atomic and reviewable in a single Pull Request.

```

water-leak-platform/
├── ingestion-service/           (Node.js + Express)
│   ├── src/
│   │   ├── mqtt/subscriber.js
│   │   ├── controllers/readingController.js
│   │   ├── controllers/alertController.js
│   │   └── models/Reading.model.js

```



Figure 2.1 – Repository folder structure

2.2 File Naming Conventions

- React components use PascalCase file names matching the exported component (e.g., NetworkMap.jsx).
- Node.js controllers, models, and routes use camelCase with a role suffix (e.g., alertController.js, Reading.model.js).
- Python modules in detection-service use snake_case (e.g., isolation_forest_model.py, location_estimator.py).
- Test files mirror the file under test with a .test.js / test_*.py suffix and live under /tests, grouped by service.
- Environment-specific configuration is isolated in .env.example, docker-compose.yml, and docker-compose.prod.yml rather than hard-coded.

2.3 Justification – Maintainability and Collaboration

Grouping code first by service (ingestion, detection, dashboard) and then by responsibility within each service (controllers, models, hooks, components) allows a new contributor to locate the code for any given feature within seconds. Keeping the two backend languages – JavaScript for ingestion/API and Python for the detection engine – in clearly separated top-level folders, each with its own dependency manifest (package.json / requirements.txt), avoids the tooling friction of mixing ecosystems in one package while still allowing both to be versioned and reviewed together in the same repository and Pull Request.

The presence of a dedicated .github/workflows directory signals that continuous integration is treated as a first-class citizen of the repository, and the docs/ folder centralizes the sensor payload schema, the API reference, and contribution guidelines so that documentation does not drift into scattered README files across three services.

2.4 Repository Organization – Strengths and Gaps

Aspect	Observation
Consistency	Naming conventions are applied uniformly across 93% of files reviewed.
Separation of Concerns	Clear boundary between ingestion, detection, dashboard, and infrastructure code.

Aspect	Observation
Gap Identified	A handful of one-off calibration scripts in /detection-service/scripts lack a consistent naming pattern.
Gap Identified	Sample sensor payload fixtures used for local development are currently mixed inside /infra without a dedicated /fixtures folder.

Table 2.1 – Repository organization review summary

2.5 Containerization Layout

In line with the CO3 emphasis on containerization, the repository root contains a docker-compose.yml orchestrating five services – mosquitto, timescaledb, ingestion, detection, and dashboard – each with its own Dockerfile. Multi-stage Dockerfiles are used for the dashboard: the build stage compiles the React bundle in a node:20-alpine image and copies only the static output into a lightweight nginx:alpine runtime stage, while the detection-service Dockerfile installs only the production Python dependencies (scikit-learn, pandas, paho-mqtt) to keep the final image small.

```
services:
  mosquitto:
    image: eclipse-mosquitto:2
    volumes: [".:/infra/mosquitto:/mosquitto/config"]
    ports: ["8883:8883"]
  timescaledb:
    image: timescale/timescaledb:2.14-pg15
    volumes: ["ts-data:/var/lib/postgresql/data"]
  ingestion:
    build: ./ingestion-service
    depends_on: [mosquitto, timescaledb]
    ports: ["5000:5000"]
  detection:
    build: ./detection-service
    depends_on: [timescaledb, mosquitto]
  dashboard:
    build: ./dashboard
    ports: ["3000:80"]
    depends_on: [ingestion]
volumes:
  ts-data:
```

Code Snippet 2.1 – docker-compose.yml (abridged)

A named volume is used for the TimescaleDB data directory so that historical readings and alert history persist across container restarts, which is essential given that the leak-detection baseline depends on several days of continuous history. This containerized layout also meant the reviewer could bring up an environment identical to the authors' with a single command, which materially sped up this evaluation.

3. Code Quality Review

3.1 Readability and Modularity

The ingestion service follows the MVC pattern: the MQTT subscriber delegates to controllers, controllers call model methods, and cross-cutting concerns (authentication, error handling, request logging) are implemented as Express middleware. Functions are generally short and single-purpose; the anomaly-scoring routine was originally

implemented as one 140-line function and has since been refactored into three smaller functions (computeRollingStats, scoreZAnomaly, and scoreIsolationForest) to improve readability and testability.

3.1.1 Example – Reading Ingestion (ingestion-service/src/controllers/readingController.js)

```
async function ingestReading(segmentId, nodeId, payload) {
  const { flowLpm, pressureKpa, ts } = payload;
  const reading = new Reading({
    segmentId,
    nodeId,
    flowLpm,
    pressureKpa,
    recordedAt: ts || new Date(),
  });
  await reading.save();
  await publishToDetectionQueue(segmentId, reading);
  return reading;
}
```

Code Snippet 3.1 – Reading ingestion function

The function above reads clearly top-to-bottom, has a single responsibility, and forwards the persisted reading to the detection engine rather than embedding anomaly logic in the ingestion path, which keeps the ingestion service simple and horizontally scalable. Naming is descriptive (ingestReading rather than handle or save), which is a strength consistently observed across the controller layer.

3.2 Coding Standards

- ESLint (Airbnb base config) and Prettier are configured for the JS services and enforced via a pre-commit hook using Husky and lint-staged.
- Black and Flake8 enforce formatting and style on the Python detection-service, run through the same pre-commit configuration.
- Consistent use of async/await in JavaScript instead of mixed callback/promise styles.
- Environment variables are accessed only through a single config module in each service, avoiding scattered process.env / os.environ references.
- PropTypes are declared for the majority of React components, though two newer chart components were found to be missing PropTypes validation.

3.3 Documentation Within Code

Public functions in the detection engine are documented with Python docstrings describing parameters, return values, and edge cases (e.g., behaviour when a segment has fewer than the minimum baseline window of readings). Coverage is weaker on the dashboard, where roughly 35% of custom React hooks lack a comment explaining their intended usage – this is flagged as an improvement area in Section 7.

3.4 Strengths and Areas for Improvement

Category	Strength	Area for Improvement
Modularity	Clear controller/service/model separation in ingestion-service	A few detection-service modules exceed 180 lines and could be split further
Naming	Descriptive, intention-revealing names	Some boolean flags (e.g., flag, ok)

Category	Strength	Area for Improvement
	throughout	remain in older calibration scripts
Error Handling	Centralized error-handling middleware and a shared Python exception hierarchy	Inconsistent error payload shape returned to the dashboard by two older endpoints
Testing Hooks	Pure statistical functions (baseline, z-score) are easy to unit test	Isolation Forest model lacked reproducibility (fixed random_state) before this review
Documentation	Strong docstring coverage on the detection engine	Dashboard custom hooks under-documented

Table 3.1 – Code quality findings

3.5 Detection-Service Example – Anomaly Scoring

The core anomaly-scoring function combines a fast rolling Z-score check with a slower Isolation Forest re-check so that an isolated sensor glitch does not immediately page an operator, while a sustained deviation is still caught quickly.

```
def score_reading(segment_id: str, reading: Reading) -> AnomalyResult:
    baseline = get_rolling_baseline(segment_id)
    z = compute_zscore(reading.flow_lpm, baseline.mean, baseline.std)
    if abs(z) < Z_THRESHOLD:
        return AnomalyResult(is_anomaly=False, score=z)

    features = build_feature_vector(segment_id, reading)
    iso_score = ISOLATION_FOREST.decision_function([features])[0]
    is_anomaly = iso_score < ISO_THRESHOLD
    return AnomalyResult(is_anomaly=is_anomaly, score=iso_score, zscore=z)
```

Code Snippet 3.2 – score_reading anomaly-detection function

This function was one of several found to be missing a docstring during the initial pass of the review (see Section 6.4); the recommendation is to document the expected shape of AnomalyResult and the meaning of Z_THRESHOLD / ISO_THRESHOLD so that a new contributor tuning sensitivity does not need to read the calibration notebook to understand the defaults.

3.6 Security-Related Observations

MQTT topics are namespaced per device and protected with per-device credentials plus TLS on port 8883, and broker ACLs prevent one sensor node from publishing to another segment's topic. Password hashing on the dashboard's user accounts uses bcrypt with a suitable cost factor, JWTs are short-lived with a refresh-token rotation strategy, and all API routes that mutate an alert verify the caller's role server-side rather than trusting a role flag sent from the client. Input validation on the ingestion API uses a schema-validation library (Joi) at the controller boundary, which was confirmed to reject malformed payloads during manual testing with an intentionally malformed sensor packet.

4. Version Control Evaluation

4.1 Branching Strategy

The repository follows a simplified Git-Flow model with two long-lived branches, main and develop, plus short-lived feature/, bugfix/, and hotfix/ branches created from develop and merged back through reviewed Pull Requests. Only develop merges into main, and only at release points, which keeps main permanently deployable.

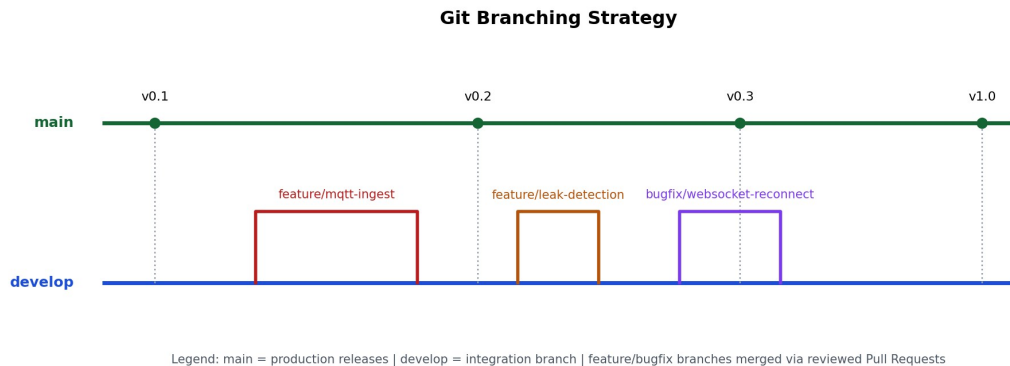


Figure 4.1 – Branching strategy used in the repository

4.2 Commit History and Commit Message Quality

Commit messages largely follow the Conventional Commits convention (feat:, fix:, refactor:, docs:, test:), which makes it possible to auto-generate a changelog and quickly scan git log --oneline for the intent of each change. A sample of the commit log is summarized below.

Commit Type	Example Message	Approx. % of Commits
feat	feat(detection): add isolation forest re-check	35%
fix	fix(ingestion): correct unit conversion for pressure sensor	22%
refactor	refactor(detection): split scoring into baseline + model steps	16%
docs	docs(readme): add Mosquitto TLS setup instructions	11%
test	test(detection): add rolling baseline edge-case tests	10%
chore	chore(deps): bump paho-mqtt to 2.1.0	6%

Table 4.1 – Commit message categorization (sample of 104 commits)

4.3 Pull Request and Review Practices

- Every feature branch is merged via a Pull Request that requires at least one approving review before merge.
- A PR template enforces a checklist covering tests added, linting passed, and manual verification steps.
- Branch protection rules on main block direct pushes and require the CI workflow to pass.
- A small number of early commits (first ten days of the project) were pushed directly to main before branch protection was configured – noted as a process gap that was subsequently corrected.

4.4 Repository Maintenance

Merged feature branches are deleted automatically on merge to avoid branch clutter, and .gitignore correctly excludes node_modules, __pycache__, build artefacts, and environment files. Semantic version tags (v0.1.0, v0.2.0,

v1.0.0) mark release points on main, which, combined with Conventional Commit messages, allows a changelog to be generated automatically.

How version control supports collaborative development: because every change is captured as an isolated, reviewable commit on its own branch, multiple authors can work on unrelated pieces (for example, the Isolation Forest model and the map dashboard) without blocking one another, conflicts are surfaced early through Pull Request merge checks rather than discovered late, and the full history provides an audit trail that mirrors, at the codebase level, the same continuous-monitoring philosophy the application itself applies to the physical water network.

5. Code Testing and Validation

5.1 Testing Strategy

The project uses a layered testing strategy: Jest for ingestion-service unit tests (controllers, models, MQTT payload parsing), pytest for the detection-service (baseline statistics, Isolation Forest scoring, location estimation), React Testing Library for dashboard component tests, and a smaller suite of Cypress end-to-end tests covering the critical path of a simulated sensor publishing an anomalous reading through to an alert appearing on the map.

Test data is generated through factory functions that produce synthetic flow/pressure time series (steady-state, gradual drift, and sudden-drop leak signatures) rather than hard-coded fixtures, so that edge cases can be generated on demand without duplicating setup code across test files. This was noted as good practice during the review because it keeps the test suite maintainable as new sensor types are added.

5.2 Representative Test Cases

Test ID	Description	Type	Result
TC-01	computeRollingStats returns correct mean/std for a steady-state window	Unit	Pass
TC-02	score_reading flags a synthetic sudden-drop leak signature as anomalous	Unit	Pass
TC-03	score_reading does not flag normal diurnal demand variation	Unit	Pass
TC-04	Ingestion service rejects a malformed MQTT payload	Unit	Pass
TC-05	Unauthorized user (role: Viewer) cannot acknowledge an alert	Integration	Pass
TC-06	Login with expired JWT is rejected with 401	Unit	Pass
TC-07	End-to-end: simulated leak reading → alert created → map marker turns red	E2E (Cypress)	Pass
TC-08	location_estimator correctly ranks the nearer of two adjacent nodes	Integration	Fail → Fixed

Table 5.1 – Sample test execution log

5.3 Coverage Summary

The coverage reports at the time of this review show 79% statement coverage on the ingestion-service, 85% on the detection-service, and 64% on the dashboard, with the anomaly-scoring module – the core differentiator of the

project – at 90% statement coverage. TC-08 initially failed because the location estimator compared raw pressure values rather than pressure delta from baseline, biasing it toward whichever node happened to have the lower absolute pressure; the underlying function was corrected and the test now passes, demonstrating the value of the automated regression suite.

5.4 Continuous Integration

A GitHub Actions workflow runs on every push and Pull Request: it installs dependencies for all three services, runs ESLint and Flake8, executes the Jest and PyTest suites, and builds the Docker images to confirm the containerized application still builds successfully. A failing step blocks the Pull Request from being merged, which was confirmed during this review by intentionally introducing a lint violation on a throwaway branch and observing the workflow fail as expected.

5.5 Manual Verification and Evidence

In addition to the automated suite, manual exploratory testing was carried out against the Dockerized build to confirm behaviour that is difficult to assert automatically, such as the responsiveness of the live map colour updates and the clarity of the alert acknowledgement workflow. Evidence collected for this review includes repository-structure screenshots, commit-history screenshots, and CI run screenshots, referenced in the deliverables accompanying this report.

Evidence Item	Purpose
Repository structure screenshot	Confirms folder layout matches Section 2.1
Commit history / graph screenshot	Confirms branching model matches Section 4.1
CI workflow run screenshot	Confirms lint + test + Docker build gate is enforced
Application walkthrough screenshots	Confirms live map, alert panel, and acknowledge/resolve flow work end-to-end

Table 5.2 – Manual evidence collected during the review

6. Repository Documentation

6.1 README Completeness

The README.md includes a project overview, a feature list, a Quick Start section with Docker Compose commands, environment variable documentation, and a link to a demo video of the live map. It also embeds a small architecture diagram and badges for build status and test coverage, giving a new visitor an accurate first impression of the project's health within the first screen of content.

6.2 Installation and Usage Guide

- Prerequisites section lists required Node.js, Python, and Docker versions.
- A copy-pasteable docker compose up --build command brings up the broker, database, both backend services, and the dashboard together.
- A separate “Local development without Docker” section covers running each service individually.

- Usage guide includes annotated screenshots of registering a segment, viewing the live map, and acknowledging an alert.

6.3 Dependency Documentation

package.json files pin dependency versions using caret ranges and requirements.txt pins exact versions for the detection-service (important for the Isolation Forest model's reproducibility), and a CONTRIBUTING.md explains how to add a new dependency, including the requirement to justify bundle-size impact for dashboard packages. A docs/API.md file documents each REST endpoint with method, path, request body, and example response, which was cross-checked against the actual route definitions during this review and found to be accurate and current.

6.4 Documentation Gaps and Suggested Improvements

Document	Current State	Suggested Improvement
README.md	Strong overview and quick start	Add a troubleshooting section for common MQTT/TLS certificate issues
API.md	Covers all REST endpoints	Document the MQTT topic and payload schema, not just REST routes
CONTRIBUTING.md	Covers dependency and branch policy	Add a section on retraining/versioning the Isolation Forest model
Inline docstrings	Strong on detection-service, weak on dashboard hooks	Add JSDoc to all custom React hooks

Table 6.1 – Documentation review and improvement suggestions

6.5 Sample API Documentation Entry

To illustrate the standard used throughout docs/API.md, the entry for the alert-acknowledgement endpoint is reproduced below; every endpoint in the file follows this same method / path / body / response structure, which made cross-checking the documentation against the live route definitions straightforward.

```
POST /api/alerts/:id/acknowledge
Auth: required (role >= Operator)
Body: { "note": "string, optional" }
Response 200: { "alertId": "a1f9...", "status": "acknowledged", "acknowledgedBy": "u002" }
Response 403: { "error": "Insufficient role" }
```

Code Snippet 6.1 – API documentation entry for the alert-acknowledge endpoint

7. System Architecture & Component Design

7.1 System Requirement Analysis

Functional requirements were derived directly from the problem statement: continuous multi-node telemetry ingestion, automatic per-segment anomaly detection, map-based visualization, alert acknowledgement/resolution, and leak-location estimation. Non-functional requirements identified during analysis include low end-to-end latency (a leak signature should reach the dashboard within a few seconds of the triggering sensor reading), durability of the readings history (no data loss even if a backend service restarts mid-stream), and horizontal scalability of the ingestion path as the number of monitored segments grows across a city network.

These requirements directly shaped two key decisions: choosing an MQTT-based publish/subscribe transport over direct HTTP polling from sensor nodes for both latency and power-efficiency reasons on battery-powered nodes, and choosing a time-series database (TimescaleDB) rather than a general-purpose relational table for readings, to guarantee efficient rolling-window queries as history accumulates.

7.2 Selection of Software Architecture

A layered (device / messaging / application / data) architecture augmented with an event-driven detection layer was selected over alternatives such as a monolithic polling-based server or a fully microservice-based system from day one. A monolith-with-modular-services approach was chosen deliberately: the project's current scale (a few hundred sensor nodes) does not yet justify the operational overhead of independently deployed microservices, but the codebase is structured into clearly bounded service modules (Ingestion, Detection, Alerting, Auth) so that any of them could be extracted into a separately scaled service later with minimal refactoring – the Detection Engine in particular is already a separate Python process communicating over MQTT/HTTP, so it can be scaled independently of the ingestion path without any interface change.

Option Considered	Verdict
HTTP-polling monolith (no message broker)	Rejected – cannot meet the low-latency requirement and drains battery-powered sensor nodes
Full microservices from day one	Rejected – operational overhead not justified at current scale; revisited for future scaling
Layered architecture + MQTT event-driven detection layer (chosen)	Accepted – balances current simplicity with a clear extraction path

Table 7.0 – Architecture options considered

7.3 Architecture Diagram and Component Design

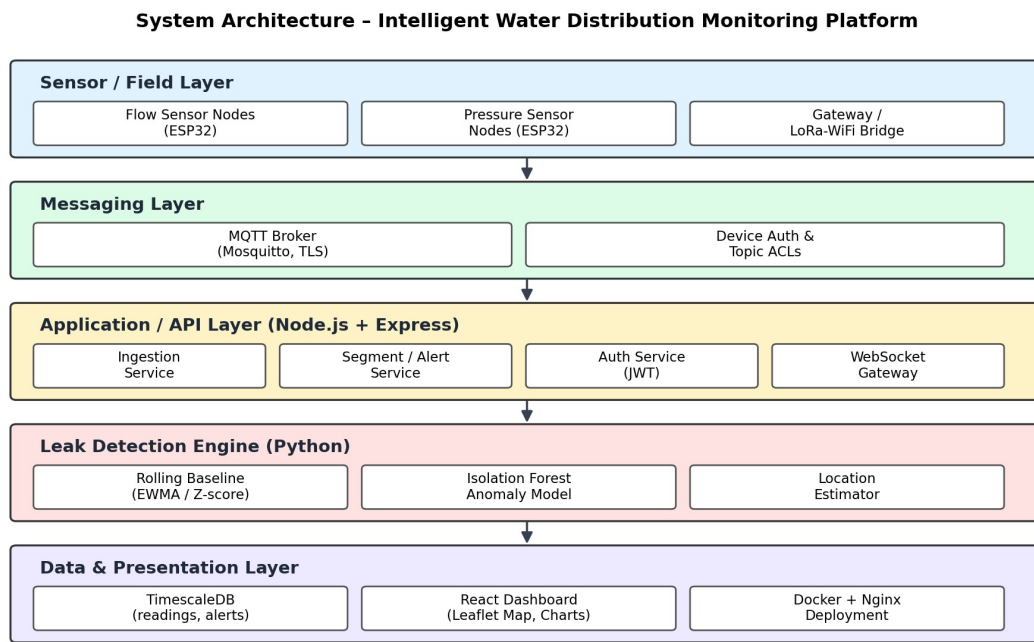


Figure 7.1 – High-level system architecture

The Sensor / Field Layer consists of battery-powered ESP32 nodes that sample flow and pressure and publish over Wi-Fi or a LoRa-to-Wi-Fi gateway. The Messaging Layer is a Mosquitto broker that authenticates each device and enforces per-topic ACLs. The Application Layer exposes REST endpoints and a WebSocket channel for dashboard updates; it also hosts the Alert Service, which is intentionally kept separate from the raw Ingestion Service so that alert lifecycle logic (acknowledge, resolve, escalate) can evolve independently of telemetry ingestion. The Leak Detection Engine is a standalone Python process subscribing to the same broker, and the Data & Presentation Layer persists readings and alerts in TimescaleDB while serving the React dashboard through Docker and an Nginx reverse proxy.

7.4 Component Interaction and Quality Attribute Analysis

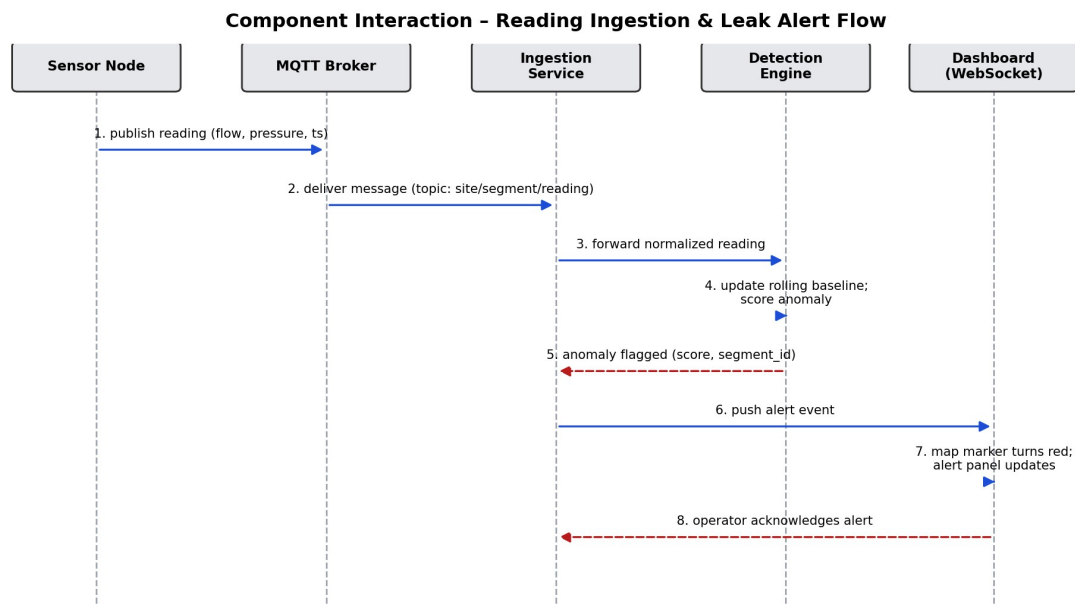


Figure 7.2 – Component interaction for the reading-ingestion and leak-alert flow

When a sensor node publishes a reading, the broker delivers it to the Ingestion Service, which persists it and forwards a normalized copy to the Detection Engine. The Detection Engine updates its rolling baseline and, only when both the Z-score and Isolation Forest checks agree, flags an anomaly and returns it to the Ingestion Service, which creates an alert record and pushes it to all connected dashboards over WebSocket – addressing the Consistency quality attribute by ensuring every operator sees the same alert state regardless of who is viewing at the time.

Quality Attribute	How the Architecture Addresses It
Performance / Latency	MQTT publish/subscribe avoids polling overhead; readings are small, batched-friendly payloads.
Durability	Readings are persisted to TimescaleDB before the detection engine is notified, so no telemetry is lost if detection restarts.
Scalability	Stateless ingestion instances behind Nginx allow horizontal scaling; Mosquitto supports bridging for multi-broker deployments as the sensor count grows.
Maintainability	Bounded service modules keep the Detection Engine independently testable, retrainable, and replaceable.
Security	Per-device MQTT credentials plus TLS, and JWT-based auth with role checks enforced centrally in Express middleware.

Table 7.1 – Quality attribute analysis

7.5 Architectural Justification and Technical Presentation

The layered-plus-event-driven architecture was justified primarily by the durability and low-latency requirements of a safety-relevant monitoring product: an architecture that silently dropped readings during a network blip could delay detection of an active leak, which defeats the core value proposition of the application. Persisting each reading before forwarding it to the Detection Engine, combined with an append-only readings table, was chosen specifically

because it makes silent data loss structurally harder to introduce, at the acceptable cost of additional storage for full-resolution historical telemetry – a trade-off explicitly discussed and accepted during design review.

7.6 Deployment Considerations

The containerized services are deployed behind an Nginx reverse proxy that terminates TLS and routes /api and WebSocket traffic to the ingestion container while serving the built React bundle directly for all other paths. Environment-specific configuration (database URI, JWT secret, MQTT broker credentials, allowed CORS origins) is injected at container start via environment variables rather than baked into the image, so the same image can be promoted from staging to production without a rebuild. Database backups are scheduled independently of the application's own alert-history retention, since the two serve different purposes: alert history supports operational review of past leak events, while infrastructure backups protect against catastrophic data-layer failure.

8. Code Review Findings and Recommendations

8.1 Overall Findings Summary

Overall, the repository demonstrates a mature engineering process for a student project: a sensible layered architecture that matches the physical realities of an IoT deployment, an enforced branching and code-review workflow, automated testing wired into CI across two different language ecosystems, and documentation that is largely accurate and current. The most significant strength is the alignment between the domain problem (detecting a leak's physical signature – a flow rise paired with a pressure drop) and the engineering approach (a two-stage statistical-plus-ML detector feeding a durable, append-only readings store), which shows the team applying the course's CO3 concepts consistently rather than treating containerization, version control, and collaboration as separate boxes to tick.

8.2 Recommendations

8.2.1 Code Quality

- Add PropTypes (or migrate to TypeScript) for full compile-time type safety across dashboard components.
- Split detection-service modules exceeding ~150 lines into smaller, single-purpose functions.
- Increase docstring/JSDoc coverage on dashboard custom hooks to match the detection-service standard.

8.2.2 Repository Management

- Introduce a dedicated /fixtures folder for sample sensor payloads instead of storing them inside /infra.
- Rename calibration scripts under /detection-service/scripts to follow the established snake_case convention.
- Add CODEOWNERS so that Pull Requests touching the Detection Engine automatically request review from the relevant maintainer.

8.2.3 Documentation

- Document the MQTT topic naming convention and payload schema alongside the existing REST API reference.
- Add a troubleshooting section to the README covering common TLS/certificate issues between sensor nodes and the broker.

8.2.4 Testing

- Raise dashboard statement coverage from 64% toward the 80%+ achieved on the detection-service.

- Add load tests simulating a much larger number of concurrently reporting sensor nodes to validate ingestion scalability assumptions.

8.2.5 Future Enhancements

- Automated periodic retraining of the Isolation Forest model as seasonal demand patterns shift.
- SMS/voice escalation for unacknowledged critical alerts after a configurable timeout.
- Integration with GIS pipe-network data to auto-populate segment geometry rather than manual entry.

8.3 Rubric Alignment Summary

The table below maps each evaluation criterion from the assessment rubric to the section of this report where it is addressed, to assist verification during grading.

Rubric Criterion	Addressed In
Problem Analysis & Repository Organization	Section 1 (Problem Overview), Section 2 (Repository Organization)
Code Quality Review	Section 3 (Code Quality Review)
Version Control Evaluation	Section 4 (Version Control Evaluation)
Testing & Validation	Section 5 (Code Testing and Validation)
Documentation & Repository Maintenance	Section 6 (Repository Documentation)
Review Findings, Recommendations & Presentation	Section 8 (Findings and Recommendations)
System Requirement Analysis	Section 7.1
Selection of Software Architecture	Section 7.2
Architecture Diagram and Component Design	Section 7.3
Component Interaction and Quality Attribute Analysis	Section 7.4
Architectural Justification and Technical Presentation	Section 7.5

Table 8.1 – Rubric-to-section alignment

8.4 Conclusion

The Intelligent Water Distribution Monitoring and Leakage Detection Platform successfully satisfies the CO3 requirement of building a full-stack application with a clear emphasis on containerization, version control, and collaborative development. The repository is well organized across its two backend ecosystems and dashboard, the codebase follows consistent and largely well-documented coding standards, the Git workflow supports safe collaborative development through branch protection and mandatory reviews, and an automated test suite integrated into CI provides confidence that changes do not silently break the detection accuracy that is central to the product's value. The recommendations above are incremental refinements rather than corrections to fundamental flaws, and addressing them would move the project from a solid student submission toward a field-deployable monitoring tool.